

# **Capstone Project Final Report**

Submitted to

Faculty of Engineering and Natural Sciences

Department of *COMPUTER ENGINEERING*

In partial fulfillment of the requirements for the degree

BACHELOR of SCIENCE in *COMPUTER ENGINEERING*

## **SkillMatch Platform: Intelligent Skill-Based Matching for Recruitment and Freelance Services**

Student: ANAS RAMADAN JUMMAH ALSHARIF - RAFAT RAED RAMEZ

ABUALHUDA – YAZAN ALI MOHAMMAD MOUSA

ID Number: 200722051 - 220901517 - 220901235

Supervisor(s): ALI GHAFARI

May, 2026

## **STUDENT DECLARATION**

By submitting this report, as partial fulfillment of the requirements of the Capstone Project, the students promise on penalty of failure of the course that

They have given credit to and declared (by citation), any work that is not their own (e.g. parts of the report that is copied/pasted from the Internet, design or construction performed by another person, etc.),

They have not received unpermitted aid for the project design, construction, report or presentation,

They have not falsely assigned credit for work to another student in the group, and not take credit for work done by another student in the group.

## Abstract

The contemporary labor market is characterized by a significant disconnect between talent availability and opportunity accessibility. While digital connectivity is ubiquitous, traditional recruitment models rely heavily on manual keyword searching and passive advertisement, leading to inefficiencies for both job seekers and employers. This project addresses these challenges by developing an Intelligent Skill-Based Matching Platform, a comprehensive digital ecosystem designed to automate the recruitment lifecycle. The proposed solution utilizes a Client-Server architecture with a Cross-Platform Mobile Application (Flutter) and a Scalable Backend (Node.js). Unlike conventional platforms, the system implements an algorithmic matching engine that analyzes candidate skills against job requirements to generate a compatibility score. Key features include real-time geolocation services for identifying local opportunities, instant messaging via WebSocket integration, and a dedicated interface for the gig economy. This report details the design and implementation of the system, including the Model-View-Controller (MVC) architectural pattern, the non-relational database schema (MongoDB), and the specific algorithms used for candidate ranking. The initial implementation demonstrates the feasibility of the platform in reducing the "experience gap" for students and streamlining the hiring process for Small to Medium Enterprises (SMEs).

Keywords: Recruitment Automation; Intelligent Matching; Flutter; Node.js; Skill Analysis.

## Table of Contents

|                                                          |    |
|----------------------------------------------------------|----|
| Abstract .....                                           | 3  |
| Table of Contents .....                                  | 4  |
| List of Tables .....                                     | 6  |
| List of Abbreviations .....                              | 6  |
| 1. Introduction.....                                     | 8  |
| 1.1 Overview .....                                       | 8  |
| 1.2 Problem Definition.....                              | 8  |
| 1.3 Project Objectives .....                             | 9  |
| 1.4 Scope and Target Market .....                        | 9  |
| 1.5 Report Organization.....                             | 9  |
| 2. Literature Review.....                                | 12 |
| 2.1 Overview of Modern Recruitment Systems.....          | 12 |
| 2.2 Analysis of Existing Solutions.....                  | 12 |
| 2.3 Theoretical Framework.....                           | 12 |
| 2.4 Comparison of Approaches.....                        | 13 |
| 3. Methodology .....                                     | 15 |
| 3.1 System Model and Data Definitions .....              | 15 |
| 3.2 Candidate Ranking and Selection Algorithm .....      | 16 |
| 3.3 Implementation Details .....                         | 17 |
| 3.4 Comprehensive Data Dictionary .....                  | 17 |
| 3.5 API Interface Specifications .....                   | 18 |
| 3.5.1 Authentication Endpoints.....                      | 18 |
| 3.5.2 Job Management and Smart Features .....            | 18 |
| 3.6 Final Methodology and Implementation Framework ..... | 19 |
| 3.7 Data and Evaluation Criteria.....                    | 19 |
| 4. Results.....                                          | 22 |

|                                                      |    |
|------------------------------------------------------|----|
| 4.1 Implemented System Components .....              | 22 |
| 4.2 Matching and Recommendation Results .....        | 22 |
| 4.3 Real-Time Communication and Smart Features.....  | 22 |
| 4.4 Verification Summary.....                        | 23 |
| 4.5 Screenshot Evidence from the Running System..... | 24 |
| 5. Discussion and Conclusion .....                   | 27 |
| 5.1 Discussion of Findings.....                      | 27 |
| 5.2 Practical Implications.....                      | 27 |
| 5.3 Limitations and Future Work.....                 | 27 |
| 5.4 Conclusion .....                                 | 28 |
| ACKNOWLEDGEMENT .....                                | 29 |
| APPENDICES .....                                     | 30 |
| Appendix A. Local Demonstration Environment .....    | 30 |
| Appendix B. Demo Accounts .....                      | 30 |
| REFERENCES .....                                     | 31 |

## List of Tables

Table 1: Comparison between Traditional Recruitment and the SkillMatch Platform

Table 2: User Entity Data Structure

Table 3: Job Entity Data Structure

Table 4: Organization Entity Structure

Table 5: Application Entity Structure

Table 6: Authentication API Routes

Table 7: Job Management API Routes

Table 8: Final Methodology Implementation Steps

Table 9: Quantitative Implementation Summary

Table 10: Implementation Verification Summary

## List of Abbreviations

| Abbreviation | Definition                        |
|--------------|-----------------------------------|
| AI           | Artificial Intelligence           |
| API          | Application Programming Interface |
| CV           | Curriculum Vitae                  |
| FCM          | Firebase Cloud Messaging          |
| JSON         | JavaScript Object Notation        |
| JWT          | JSON Web Token                    |
| MVC          | Model-View-Controller             |
| NoSQL        | Non-Relational Database           |
| REST         | Representational State Transfer   |
| SME          | Small to Medium Enterprise        |
| UI           | User Interface                    |
| URL          | Uniform Resource Locator          |

# **Section 1:**

## **Introduction**

## 1. Introduction

### 1.1 Overview

The contemporary labor market is characterized by a paradox: while digital connectivity is at an all-time high, the process of matching talent with opportunity remains inefficient. Many individuals, particularly students, fresh graduates, and freelancers, face significant barriers in securing work or training that aligns with their specific skill sets. Concurrently, small to medium enterprises (SMEs) and companies encounter substantial challenges in accessing suitable candidates quickly and cost-effectively [1], [2].

This project introduces a digital platform designed to bridge this gap through an Intelligent Skill-Based Matching System. Unlike traditional recruitment models that rely on manual searching and passive filtering, the proposed platform utilizes algorithmic matching to automatically link users based on their competencies. This approach aims to facilitate recruitment, training, and freelance contracting by shifting the focus from "years of experience" to "verified skills."

### 1.2 Problem Definition

The current recruitment landscape suffers from systemic inefficiencies that affect both job seekers and employers. These challenges can be categorized into three primary areas:

**Inefficiencies in Manual Search:** Most existing job platforms rely heavily on manual search mechanisms. Candidates are required to filter through broad listings, while companies must sift through high volumes of applications. This traditional approach is time-consuming and often results in a mismatch between the candidate's actual capabilities and the job requirements [2].

**The Passive Hiring Trap:** In the prevailing model, companies typically post an advertisement and wait passively for applicants to respond. This reactive approach delays the hiring process and does not guarantee access to the best available talent, as high-quality candidates may not see specific advertisements during their active period.

**Structural Barriers for Early-Career Professionals:** A major deficiency in the current market is the exclusion of individuals entering the workforce. Recruitment algorithms often prioritize candidates with extensive professional history, creating an "experience gap" for students and graduates who possess theoretical knowledge but lack the specific years of experience required by search filters. Furthermore, the rise of the gig economy has not been fully met by



integrated platforms, leaving freelancers without specialized tools to market short-term services effectively

### *1.3 Project Objectives*

The primary objective of this project is to develop a comprehensive platform that automates the recruitment process through smart matching. The specific goals are as follows:

**Automation of Recruitment:** To replace the traditional manual search process with a system that analyzes user skills and automatically links them to suitable opportunities, thereby saving time for both individuals and companies.

**Instant Candidate Retrieval:** To enable companies to obtain a curated list of the best-fit candidates immediately, removing the latency associated with posting ads and waiting for applications.

**Integration of the Gig Economy:** To support freelance work and short-term projects with the same level of infrastructure as full-time employment, allowing individuals to advertise specific micro-services (e.g., logo design, content writing).

**Empowerment of Underrepresented Groups:** To improve the employment prospects for students and new graduates by providing a system that highlights practical skills and matches them with training programs, rather than filtering them out based on lack of prior tenure.

### *1.4 Scope and Target Market*

The scope of this project extends beyond standard corporate recruitment to address the specific needs of emerging markets, particularly in the Middle East and Africa. In these regions, the youth population faces distinct challenges in finding direct work opportunities. The platform is specifically designed to bridge the gap between local talent and the needs of Small to Medium Enterprises (SMEs). The system encompasses three distinct operational domains: full-time recruitment, internship placement for students, and freelance service contracting.

### *1.5 Report Organization*

The remainder of this report is organized as follows:

Section 2 (Literature Review) analyzes existing recruitment platforms and discusses the theoretical framework behind smart matching, cross-platform development, real-time communication, and document-oriented storage.

Section 3 (Methodology) describes the system model, data definitions, matching algorithm, API structure, final implementation framework, data/case definition, and evaluation criteria.

Section 4 (Results) presents the implemented system components, matching and recommendation results, real-time features, smart job creation, quantitative implementation summary, and local verification outcomes.

Section 5 (Discussion and Conclusion) discusses the main findings, practical implications, limitations, future work, and the final conclusion of the project.

# **Section 2:**

## Literature Review

## 2. Literature Review

### *2.1 Overview of Modern Recruitment Systems*

The digitization of recruitment has transformed how organizations identify talent. Early e-recruitment systems were essentially digital bulletin boards, where employers posted static advertisements and candidates submitted disparate documents. However, as the volume of applications has grown, these traditional methods have revealed significant scalability issues. Research indicates that manual filtering of resumes is not only time-intensive but also prone to cognitive bias, often leading to suboptimal hiring decisions. The current trend in literature emphasizes a shift towards "Smart Recruitment," which leverages data processing and algorithmic matching to reduce human intervention and increase precision.

### *2.2 Analysis of Existing Solutions*

To understand the necessity of the proposed platform, it is critical to analyze the limitations of current market leaders, particularly LinkedIn and traditional job boards.

**Corporate Focus and Rigidity:** Platforms such as LinkedIn are optimized for large-scale corporate structures and long-term employment. Their algorithms prioritize candidates with established professional networks and extensive tenure, effectively alienating students, fresh graduates, and blue-collar workers who lack a digital footprint.

**Communication Bottlenecks:** Existing platforms often rely on asynchronous, formal messaging systems (e.g., email-like inboxes). This structure creates a communication bottleneck where scheduling simple interviews becomes a multi-step process, unsuited for the gig economy or urgent hiring needs.

**Lack of Geolocation Features:** For many sectors, particularly in service industries and internships, physical proximity is a deciding factor. Most current platforms lack intuitive mapping features that allow users to visualize opportunities in their immediate vicinity, forcing users to rely on text-based location filters that are often inaccurate.

### *2.3 Theoretical Framework*

The proposed "SkillMatch Platform" is built upon several key technological frameworks that address the limitations identified above.

**Client-Server Architecture (MVC Pattern):** The system adopts the Model-View-Controller (MVC) architectural pattern [6], [8] to ensure separation of concerns [6]. In this model, the View (Flutter) handles the user interface, the Controller (Node.js/Express) manages business

logic and data orchestration [4], [7], and the Model (Mongoose) defines the data structure. This separation allows for scalable development where the backend logic remains independent of the mobile interface.

Cross-Platform Mobile Development (Flutter): To maximize accessibility, the client side is developed using Flutter, a UI toolkit that compiles to native code for both Android and iOS [3]. Unlike web-based wrappers, Flutter allows for direct interaction with device hardware, which is essential for the platform's real-time geolocation and video calling features.

Non-Blocking I/O (Node.js): Given the requirement for real-time matching and chat, the backend utilizes Node.js. Its event-driven, non-blocking I/O model is specifically suited for data-intensive real-time applications (DIRT), allowing the server to handle thousands of concurrent connections (e.g., chat WebSocket streams) without thread blocking [4].

Document-Oriented Database (MongoDB): Recruitment data is inherently unstructured; user profiles vary significantly in length, and skill sets differ across industries. We utilize MongoDB, a NoSQL database, to store this data in flexible JSON-like documents. This allows for dynamic profile updates without the rigid schema constraints of traditional SQL databases [5].

## 2.4 Comparison of Approaches

Table 1 summarizes the functional differences between traditional recruitment methods and the proposed intelligent platform.

**Table 1: Comparison between Traditional Recruitment and the SkillMatch Platform**

| Feature          | Traditional Platforms (e.g., LinkedIn) | SkillMatch Platform              |
|------------------|----------------------------------------|----------------------------------|
| Matching Logic   | Keyword search & Manual filtering      | Algorithmic Skill-Based Matching |
| User Interaction | Passive (Post & Wait)                  | Active (Instant Retrieval)       |
| Target Audience  | Corporate Professionals                | Students, Freelancers, SMEs      |
| Communication    | Formal Email/InMail                    | Real-time Chat & Video           |
| Location Logic   | Text-based City Filters                | Geolocation & Interactive Map    |

# **Section 3:**

## Methodology

### 3. Methodology

#### 3.1 System Model and Data Definitions

The proposed platform operates on a data-centric model where entities are defined by strict schemas to ensure consistency during the matching process. We define the system  $SS$  as a tuple  $SS = \{U, J, M\}$ , where  $U$  represents the set of Users,  $J$  represents the set of Job Postings, and  $M$  represents the Match Scores derived from semantic analysis.

**User Entity Definition:** The User entity ( $U$ ) is the fundamental unit of the platform. As defined in the User.js schema, each user  $u \in U$  is characterized by a set of attributes including role-based access control and real-time status indicators. The formal definition of the User object is presented in Table 2.

**Table 2: User Entity Data Structure**

| Attribute  | Data Type     | Description                                                                                    |
|------------|---------------|------------------------------------------------------------------------------------------------|
| id         | ObjectId      | Unique identifier for the user.                                                                |
| role       | Enum          | Classification of user type: ["admin", "Organization", "Freelancer", "Job Seeker"].            |
| analyzed   | String        | The parsed text data extracted from the uploaded PDF resume, used for algorithmic matching.    |
| fcmTokens  | Array[String] | Firebase Cloud Messaging tokens used for targeting push notifications to specific devices [9]. |
| socketIds  | Array[String] | Active socket connections for real-time chat status (online/offline).                          |
| isVerified | Boolean       | Security flag to ensure trust in the ecosystem.                                                |

**Job Entity Definition:** The Job entity ( $J$ ) represents the demand side of the recruitment equation. As defined in Job.js, each job  $j \in J$  contains specific metadata required for the matching algorithm.

**Table 3: Job Entity Data Structure**

| Attribute        | Data Type     | Description                                                                             |
|------------------|---------------|-----------------------------------------------------------------------------------------|
| title            | String        | The designation of the vacancy.                                                         |
| jobType          | Enum          | The contractual nature: ['Full-Time', 'Part-Time', 'Remote', 'Internship', 'Contract']. |
| requirements     | Array[String] | List of mandatory skills or qualifications.                                             |
| responsibilities | Array[String] | List of expected duties.                                                                |

|           |          |                                                             |
|-----------|----------|-------------------------------------------------------------|
| companyId | ObjectId | Reference to the Organization entity that owns the posting. |
|-----------|----------|-------------------------------------------------------------|

### 3.2 Candidate Ranking and Selection Algorithm

A core contribution of this project is the Ranking and Selection Mechanism, which filters candidate profiles based on a pre-calculated suitability score ( $SS_{\text{match}}$ ).

The ranking process is handled by the jobMatchController module. Unlike simple keyword matching, this controller implements a threshold-based filtering logic. The system enforces a minimum relevance threshold ( $\theta = 50\%$ ) to ensure that employers are only presented with viable candidates. Matches falling below this threshold are discarded to reduce noise.

The selection logic operates as follows:

**Authentication:** The system validates the requesting organization's identity ( $Org\_ID$ ) using the security token.

**Retrieval:** The system queries the JobMatch collection for all potential candidates associated with the target Job ID ( $J\_ID$ ).

**Filtering:** It applies a filter where the match score  $SS_{\text{match}} > 50$ .

**Sorting:** The results are sorted in descending order ( $SS_{\text{match}} \searrow$ ), ensuring the best-fit candidates appear first.

**Population:** The candidate's full profile details (from the User collection) are joined with the match record for presentation.

The pseudo-code for this process is formally defined in Algorithm 1.

#### Algorithm 1: Candidate Retrieval and Ranking Strategy

Input:  $Req$  (Request Object containing User ID),  $J\_ID$  (Target Job ID)

Output:  $L_{\text{ranked}}$  (List of sorted candidate profiles)

```

BEGIN
   $Org\_ID \leftarrow Req.user.id$ 
   $Organization \leftarrow FindById(Org\_ID)$ 
  IF  $Organization$  is NULL THEN
    **RETURN** Error("Unauthorized Access")
  END IF

   $Matches \leftarrow Query(JobMatch)$ 
  WHERE  $jobId == J\_ID$ 

```



```

AND $matchScore > 50$
IF $Matches$ is EMPTY THEN
  **RETURN** Error("No matched users found above threshold")
ELSE
  $L_{ranked} \leftarrow$ **Sort**($Matches$) by $matchScore$ **DESCENDING**
  $L_{final} \leftarrow$ **Populate**($L_{ranked}$, 'userId')
  **RETURN** $L_{final}$
END IF

END

```

### 3.3 Implementation Details

The system logic is implemented using Node.js to ensure non-blocking performance. The `getMatchSortedByScore` function serves as the primary endpoint for the matching interface. It leverages Mongoose's `populate()` method to perform a relational join between the Match Data and User Data in a single query execution, optimizing database performance and reducing latency.

### 3.4 Comprehensive Data Dictionary

To ensure data integrity, the system utilizes a strict schema definition for all entities. Beyond the primary User and Job entities, the platform manages Organizations and Applications using the following specifications.

**Organization Entity:** The Organization model represents the corporate entities or SMEs posting vacancies. Unlike standard users, this entity includes specific fields for industry classification and followers to support the social networking aspect of the platform.

**Table 4: Organization Entity Structure**

| Attribute  | Data Type     | Description                                               |
|------------|---------------|-----------------------------------------------------------|
| name       | String        | Legal name of the company (Unique).                       |
| industry   | String        | Sector classification (e.g., "Software", "Construction"). |
| websiteURL | String        | Optional link to the company's external site.             |
| address1   | String        | Primary physical address for geolocation.                 |
| followers  | Array[String] | List of User IDs subscribed to company updates.           |
| fcmTokens  | Array[String] | Firestore Cloud Messaging tokens for push notifications.  |
| isVerified | Boolean       | Verification to establish trust.                          |

Application Entity: The Application model tracks the lifecycle of a candidate's submission. Notably, this schema utilizes denormalization (storing jobTitle and userName directly) to reduce database join operations when retrieving application lists.

**Table 5: Application Entity Structure**

| Attribute      | Data Type | Description                                                             |
|----------------|-----------|-------------------------------------------------------------------------|
| userId         | ObjectId  | Reference to the User (Candidate).                                      |
| organizationId | ObjectId  | Reference to the User (Employer).                                       |
| jobId          | ObjectId  | Reference to the Job (Vacancy).                                         |
| matchScore     | Number    | The AI-calculated suitability score (0-100) at the time of application. |
| appliedDate    | Date      | Timestamp of submission (Default: Date.now).                            |
| status         | String    | Implicitly managed via business logic.                                  |

### 3.5 API Interface Specifications

The backend exposes a structured RESTful API. The routing layer is organized by resource type, ensuring a clean separation of endpoints.

#### 3.5.1 Authentication Endpoints

The authRoutes.js module handles secure entry into the system. It supports standard login/register flows as well as email-based verification and password resets.

**Table 6: Authentication API Routes**

| Method | Endpoint                      | Description                                                            |
|--------|-------------------------------|------------------------------------------------------------------------|
| POST   | /api/auth/register            | Registers a new user (Admin, Organization, Freelancer, or Job Seeker). |
| POST   | /api/auth/login               | Authenticates credentials and returns a JWT bearer token.              |
| GET    | /api/auth/verify-email/:token | Verifies the user's email address via a token sent by the system.      |
| POST   | /api/auth/forgot-password     | Initiates the password recovery flow via email.                        |
| POST   | /api/auth/verify-reset-code   | Validates the OTP code sent to the user for password reset.            |
| POST   | /api/auth/set-new-password    | Finalizes the password change process.                                 |

#### 3.5.2 Job Management and Smart Features

The jobsRoutes.js module manages the lifecycle of job postings. A key innovation here is the smart-add-job endpoint, which utilizes AI to parse file uploads into structured job data.

### 3.6 Final Methodology and Implementation Framework

The final methodology follows a design-and-implementation framework. The research problem is addressed by building a working software prototype and evaluating whether its components satisfy the project objectives: skill-based matching, role-based recruitment workflows, real-time communication, smart job creation, and support for freelance opportunities. The final system uses candidate profile data, job posting data, and interaction data as the basis for matching and recommendation behavior.

The implementation was divided into sequential steps beginning with data modeling, followed by backend API development, frontend interface construction, matching workflow implementation, real-time communication support, and local verification. Table 8 summarizes the final implementation steps used in the project.

**Table 8: Final Methodology Implementation Steps**

| Step | Methodological Activity                  | Implemented Component                                                                                                                 |
|------|------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| 1    | Define system entities and relationships | Mongoose models for users, organizations, jobs, applications, skills, messages, ratings, and notifications                            |
| 2    | Develop backend services                 | Node.js and Express REST API routes for authentication, jobs, applications, messaging, meetings, freelance posts, and admin functions |
| 3    | Develop frontend workflows               | Flutter interfaces for job seekers, freelancers, organizations, and administrators                                                    |
| 4    | Implement matching logic                 | JobMatch scoring workflow, ranked job display, and organization-side candidate retrieval                                              |
| 5    | Support real-time interaction            | Socket.IO namespaces for presence, chat, and call-related signaling                                                                   |
| 6    | Verify local demonstration               | Run scripts, health endpoint, seeded demo accounts, and web build served locally                                                      |

### 3.7 Data and Evaluation Criteria

The project uses a demonstration case based on a skill-based recruitment platform for students, freelancers, job seekers, and organizations. Demo accounts and seed data were prepared for the main roles so the platform can be tested without requiring evaluators to manually create all records. The main evaluation criteria are whether the implemented

platform can authenticate different roles, create and retrieve job postings, calculate and store match scores, sort jobs and candidates by relevance, support real-time messaging, and launch successfully in a local demonstration environment.

# **Section 4:**

## Results

## 4. Results

### *4.1 Implemented System Components*

The implementation phase of the SkillMatch Platform was completed as a working client-server application. The final system integrates a Flutter-based cross-platform client, a Node.js and Express backend, MongoDB data models, JWT-based authentication, WebSocket communication, Firebase Cloud Messaging support, and external AI services for selected smart features.

The completed implementation includes four major user roles: Administrator, Organization, Freelancer, and Job Seeker. The authentication module supports user registration, login, email verification, password reset, and token-based session management. Role-specific interfaces were implemented so that organizations can manage vacancies and applicants, job seekers can browse and apply for opportunities, freelancers can publish services, and administrators can monitor platform activity.

### *4.2 Matching and Recommendation Results*

The intelligent matching feature was implemented through a scoring workflow that compares candidate profile information with job requirements and stores the resulting score in the JobMatch collection. For job seekers, the platform retrieves available jobs and sorts them by match score so that the most relevant opportunities appear first. For organizations, the candidate retrieval mechanism filters match records using a threshold of  $S\_match > 50$  and returns matched candidates in descending score order.

### *4.3 Real-Time Communication and Smart Features*

Socket.IO namespaces were implemented for presence, chat, and call-related signaling. Users can register active socket sessions, exchange messages, receive delivery feedback, and update online status. Firebase Cloud Messaging integration was also added to support push notifications for chat, job updates, meetings, and applicant activity, depending on valid Firebase configuration and device tokens.

The smart job creation feature allows organizations to create structured job postings from text input or supported uploaded files. The backend extracts job information and uses an AI-assisted parsing workflow to produce fields such as title, description, location, salary, job type, category, deadline, requirements, and responsibilities.

Quantitatively, the implementation contains 13 backend data models, 105 backend API route definitions, 14 controller modules, 15 route modules, and 136 Dart source files. These values show that the project is not limited to a single demonstration screen; it includes multiple integrated workflows across data storage, backend services, frontend user interfaces, and real-time interaction. The strongest implemented areas are authentication, profile and skills management, job posting, matching, messaging, notifications, and role-based dashboards.

**Table 9: Quantitative Implementation Summary**

| Metric                        | Observed Value                    | Interpretation                                                                                                                                                           |
|-------------------------------|-----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Backend data models           | 13 Mongoose models                | Covers users, organizations, jobs, applications, skills, messaging, meetings, ratings, notifications, and freelance data.                                                |
| Backend API route definitions | 105 route definitions             | Shows broad REST coverage across authentication, job management, applications, profiles, posts, messages, skills, ratings, notifications, meetings, and admin functions. |
| Backend modular structure     | 14 controllers and 15 route files | Indicates separation of business logic and routing concerns.                                                                                                             |
| Frontend source coverage      | 136 Dart source files             | Represents role-specific Flutter screens, services, models, utilities, and shared widgets.                                                                               |
| Supported user roles          | 4 roles                           | Administrator, Organization, Freelancer, and Job Seeker workflows are represented.                                                                                       |
| Local verification services   | 2 local services                  | Backend API on port 5000 and Flutter web app on port 5050 were verified locally.                                                                                         |

#### 4.4 Verification Summary

Local verification confirmed that the backend API and Flutter web build can be launched from the project run scripts. The backend exposes a health endpoint at `/api/health`, while the web application is served locally for demonstration. Table 10 summarizes the main verification evidence collected during review.

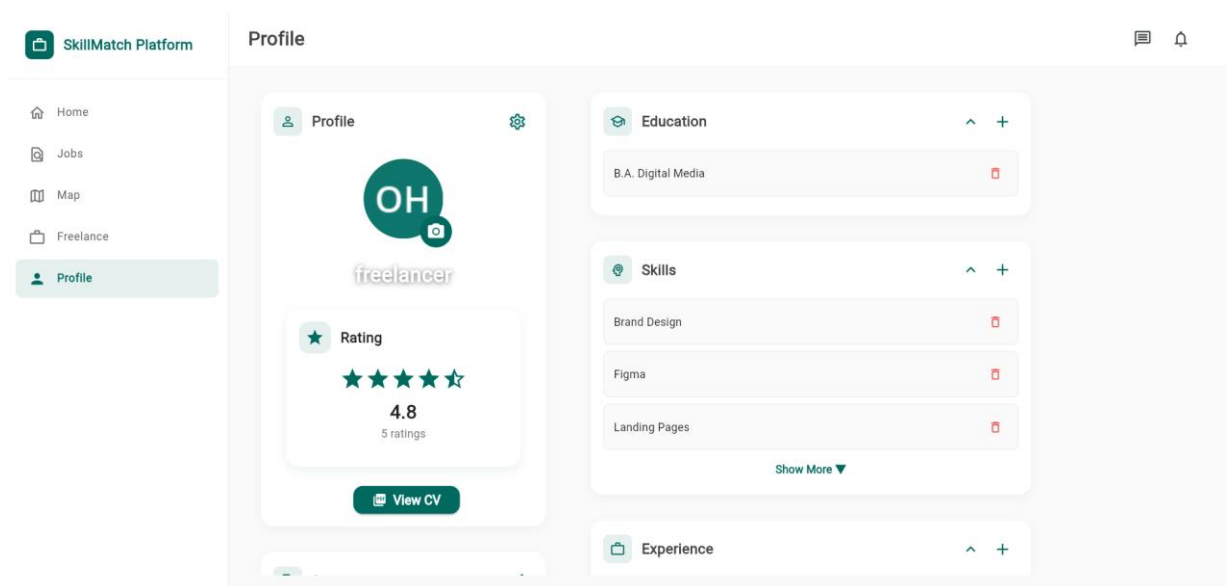
**Table 10: Implementation Verification Summary**

| Area                     | Evidence                                                      | Result                             |
|--------------------------|---------------------------------------------------------------|------------------------------------|
| Backend syntax           | Node.js syntax check across backend JavaScript files          | Passed                             |
| Frontend static analysis | Flutter analyze executed on the frontend project              | Completed with lint/style warnings |
| Local backend launch     | Backend health endpoint responded at <code>/api/health</code> | Passed                             |
| Local frontend launch    | Flutter web build served at                                   | Passed                             |

|                     |                                                                     |                        |
|---------------------|---------------------------------------------------------------------|------------------------|
|                     | http://localhost:5050                                               |                        |
| Database dependency | MongoDB Atlas connection requires valid network access/IP whitelist | Configuration required |
| Document package    | Final DOCX opens successfully in Microsoft Word                     | Passed                 |

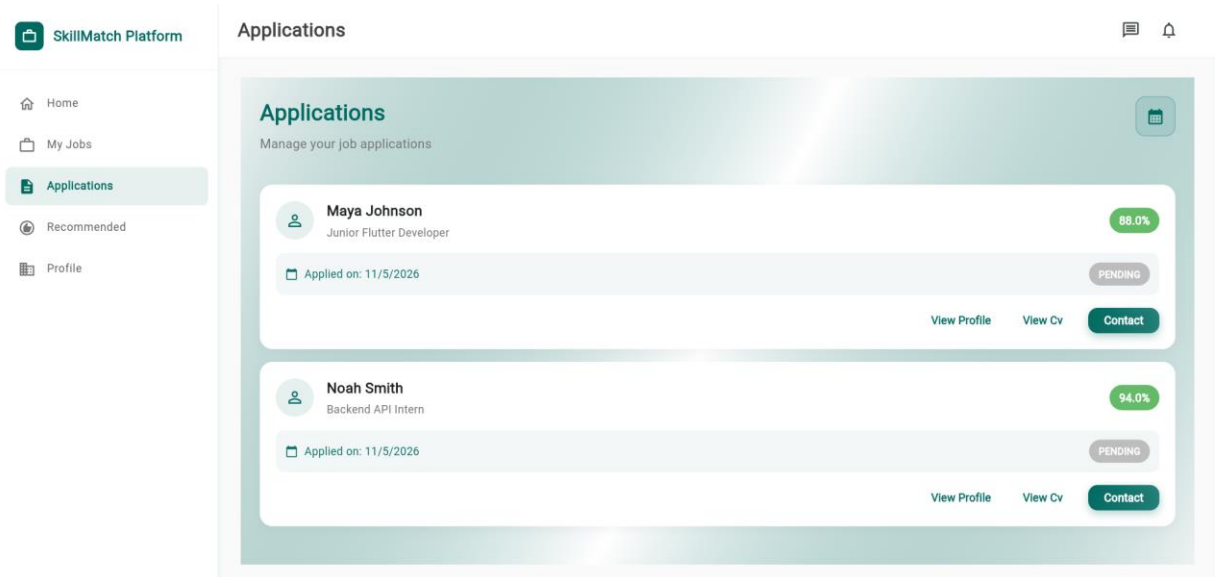
#### 4.5 Screenshot Evidence from the Running System

The screenshots below provide visual evidence from the running web application after the demo data was prepared in MongoDB. They show the job seeker profile/CV workflow, organization-side application review, and administrator monitoring dashboard.



*Screenshot 1. Job seeker profile and CV evidence for Maya Johnson after uploading the demo resume.*





*Screenshot 2. Organization applications view showing applicants and match scores for posted jobs.*



*Screenshot 3. Administrator statistics dashboard showing platform-level users, jobs, posts, and activity indicators.*

Overall, the implementation results show that the project achieved its main objectives: automating parts of recruitment, improving candidate and job discovery, supporting real-time communication, enabling freelance activity, and reducing barriers for students and early-career professionals.

# **Section 5:**

## Discussion and Conclusion

## 5. Discussion and Conclusion

### *5.1 Discussion of Findings*

The results indicate that the platform improves the recruitment workflow in three main ways. First, it reduces manual effort by ranking jobs and candidates according to match scores. Second, it improves communication speed through real-time chat, presence, and notification features. Third, it expands the system beyond standard full-time recruitment by including freelancers, students, and early-career users as first-class participants in the platform.

The selected technologies were appropriate for the project requirements. Flutter allowed the development of a unified interface for web and mobile-oriented use cases. Node.js and Express provided a flexible backend for REST APIs and real-time Socket.IO communication. MongoDB was suitable for storing flexible recruitment data, since user profiles, skills, experience, applications, and freelance posts can vary significantly between users.

### *5.2 Practical Implications*

A major practical implication of the project is its focus on accessibility for underrepresented users in the labor market. Students, fresh graduates, and freelancers often struggle because traditional platforms prioritize professional history and existing networks. SkillMatch instead emphasizes skills, profile attributes, and opportunity relevance, which can help users with limited formal experience become more visible to organizations.

### *5.3 Limitations and Future Work*

Despite the successful implementation, several limitations remain. The current system is a capstone prototype rather than a large-scale production deployment. Performance was verified in a local and demo-data environment, but the platform has not yet been stress-tested with thousands of simultaneous users. In addition, the matching workflow relies on stored scores and AI-assisted scoring logic; future work could improve this by training dedicated recommendation models on larger datasets.

Future improvements should focus on stronger recommendation accuracy, production deployment, and evaluation with real users. A machine learning recommendation model could be trained using historical applications, hiring outcomes, profile quality, and employer feedback. The platform could also include organization analytics dashboards, improved fraud detection, multilingual support, advanced search filters, interview scheduling, and cloud-native deployment with monitoring and automated backups.

#### *5.4 Conclusion*

In conclusion, the SkillMatch Platform successfully demonstrates how intelligent automation can modernize recruitment and freelance service discovery. The project delivers a working technical foundation that combines skill-based matching, role-based workflows, real-time communication, notification support, geolocation features, and smart job creation. While further production hardening and larger-scale testing are needed, the system meets the main objectives of the capstone project and provides a strong foundation for future development into a scalable employment platform.

## **ACKNOWLEDGEMENT**

I would like to express my sincere gratitude to my supervisor, ALI GHAFARI, for the guidance, feedback, and support provided throughout the development of this capstone project. I also extend my appreciation to the Faculty of Engineering and Natural Sciences for providing the academic environment and resources necessary to complete this work. Finally, I would like to thank everyone who supported the development, testing, and refinement of the SkillMatch Platform.

## APPENDICES

### *Appendix A. Local Demonstration Environment*

Backend API: `http://localhost:5000`

Frontend web application: `http://localhost:5050`

Backend health endpoint: `http://localhost:5000/api/health`

Run command: `.\run-skillmatch.ps1`

### *Appendix B. Demo Accounts*

All demo accounts use the password 123456.

Admin: `admin@admin.com`

Organization: `organization@demo.com`

Job Seeker: `jobseeker@demo.com`

Freelancer: `freelancer@demo.com`

Additional users: `developer@demo.com`, `designer@demo.com`, `healthorg@demo.com`

## REFERENCES

- Express.js Team. (2024). Express: Fast, unopinionated, minimalist web framework for Node.js. <https://expressjs.com>.
- Firebase. (2024). Firebase Cloud Messaging architectural overview. Google. <https://firebase.google.com/docs/cloud-messaging>.
- Flutter Team. (2024). Flutter: Build apps for any screen. Google Developers. <https://flutter.dev>.
- Fowler, M. (2002). Patterns of enterprise application architecture. Addison-Wesley Professional.
- Martin, R. C. (2017). Clean architecture: A craftsman's guide to software structure and design. Prentice Hall.
- MongoDB Inc. (2024). The MongoDB manual: Document-oriented storage. <https://docs.mongodb.com/manual>.
- OpenJS Foundation. (2024). Node.js JavaScript runtime documentation. <https://nodejs.org/en/docs>.
- Okatta, C. G., Ajayi, F. A., & Olawale, O. (2024). Navigating the future: Integrating AI and machine learning in HR practices for a digital workforce. *Computer Science & IT Research Journal*, 5(4). <https://doi.org/10.51594/csitj.v5i4.1085>.
- Palos-Sanchez, P. R., Baena-Luna, P., Badicu, A., & Infante-Moro, J. C. (2022). Artificial intelligence and human resources management: A bibliometric analysis. *Applied Artificial Intelligence*, 36(1). <https://doi.org/10.1080/08839514.2022.2145631>.